COMP 416
OCM416: Open Charge Map-API Based Application Layer Protocol

Saliha Zülal Demirtaş 79747

This project implements a client-server application for retrieving information about the nearest electrical car charging stations using Open Charge Map (OCM) API. The communication between the client and the server is based on a custom OCM416 protocol.

Server is a multithreaded server; so, multiple clients can communicate with the server simultaneously. The server validates the inputs, sends the appropriate status code in a specific format, and the comma-separated response. The client is a command-line program that allows the user to select the desired action from the menu, and displays the message in a user-friendly format.

Two separate Java projects are implemented for the Server and the Client.

Workflow

1. Client sends a request based on the input given by the user: OCM416 LOCS
2. Server validates the request and parses arguments.
3. Server calls OCMApiHandler, constructs an HTTP request using the Java HttpClient, sends it to the OCM API, parses the JSON response, and returns the formatted data.
4. Server sends the status message and response message in separate lines.
5. Client retrieves the data and displays it in a readable format.

OCM Server Overview

The implementation of the server is divided into three packages: api, data, and server.

Server Package

OCMServer

The main server opens a ServerSocket on the specified port. For each accepted client, a new OCMServerThread is created which manages one client session. The thread is generated inside OCMServer as below.

```
private void listenAndAccept()  1 usage
{
    System.out.println("OCM Server started on port: " + port);
    while (serverSocket != null && !serverSocket.isClosed()) try {
        Socket clientSocket = serverSocket.accept();
        OCMServerThread st = new OCMServerThread(clientSocket);
        st.start();
    } catch (Exception e) {
        System.err.println(e.getMessage());

        if (serverSocket == null || serverSocket.isClosed()) {
            break;
        }
    }
}
```

OCMServerThread

When a client is accepted by OCMServer, OCMServerThread is created, which inherits the Thread class. It runs in parallel for each client. It creates an **OCMClientHandler** instance, and it passes the socket to OCMClientHandler for handling the communication with the client. It overrides the run() method to print an information message and to create an OCMClientHandler.

```
@Override
public void run() {
    System.out.println("A connection was established with a client on the address of " + socket.getRemoteSocketAddress());
    OCMClientHandler handler = null;
    try {
        handler = new OCMClientHandler(socket);
        handler.startSession();
    } catch (Exception e) {
        System.err.println(e.getMessage());
    }
}
```

OCMClientHandler

This class is the core of the server logic since it implements the OCM416 protocol by reading client messages and validating them, fetching the data, and returning appropriate status messages. It reads the client message and parses the protocol format according to the OCM416 string and the flags in the message.

It handles four commands:

- **STATIONS:** To retrieve the nearest charging stations based on the location, country, or the coordinates given as parameters. The OCMClientHandler detects conflicting or invalid parameters.
- **LOCS:** To display the locations stored.

- **CTS:** To display the supported countries.
- **EXT:** To close the session.

It returns the response and appropriate status code from the following dictionary:

```
public Map<StatusCode, String> statusCodes = Map.of(  1 usage
        StatusCode.OCM_200,  v1: "Success",
        StatusCode.OCM_400,  v2: "Bad Request",
        StatusCode.OCM_404,  v3: "Not Found",
        StatusCode.OCM_500,  v4: "Internal Server Error"
);
```

- OCM_200 is sent when input is valid and data is available. OCM_200 is sent even if the data is empty but valid.
- OCM_400 is sent when parameters are invalid or missing.
- OCM_404 is sent when a country or location is not found.
- OCM_500 is sent when there is an API failure or network failure.

The server sets a one-minute read timeout on each client socket. If no client request arrives in a specified time period, a SocketTimeException is thrown.

```
OCM Server started on port: 8000
A connection was established with a client on the address of /127.0.0.1:55652
Client timed out.
```

The client side is also closed, when the user attempts to perform an action.

```
Select the method
a. By coordinates - Enter latitude and longitude
b. By country - Enter country name
c. By location - Enter location name
d. Go back
>2
Connection Closed
> Server closed the connection.
Stream closed
```

Api Package

OCMApiHandler

This class connects to the public OCM API by using the API endpoints and the private user key, and retrieves the required information.

The base url is "https://api.openchargemap.io/v3" Initially, the country list is fetched from /referencedata endpoint of the API as the following, and the countryMap is initialized by getCountryData() method in the beginning of the program. HttpRequest is created in the following format. The response is then retrieved and parsed using the methods from ResponseParser class.

```
String url = baseUrl + "/referencedata";

HttpRequest request = HttpRequest.newBuilder()
        .header( name: "Accept",  value: "application/json")
        .header( name: "X-API-Key", apiKey)
        .uri(URI.create(url))
        .GET()
        .build();
```

The stations are fetched from /poi endpoint by querying using the given parameters such as latitude, longitude, country code, or location.

OCMResponseParser

It uses Gson to parse JSON responses sent from the API into Java classes such as **OCMCountryData** or **OCMStationData.**

Data Package

OCMCountryData

It holds a country class, consisting of Title and ISOCode, and Countries array storing countries.

```
public class OCMCountryData {  7 usages

    public Country[] Countries;  1 usage

    public static class Country {  4 usages
        public String Title;  1 usage
        public String ISOCode;  1 usage
    }
}
```

OCMStationData

It holds the address specific data and the number of charging points.

```
public class OCMStationData {  3 usages

    public int NumberOfPoints;  1 usage
    public AddressData AddressInfo;  4 usages

    public static class AddressData {  1 usage
        public String Title;  1 usage
        public String AddressLine1;  1 usage
        public double Latitude;  1 usage
        public double Longitude;  1 usage
    }

}
```

OCMLocationData

It stores the predefined locations with coordinates.

```
public static Map<String, Location> LocationsMap = new HashMap<>();  13 usages
static {
    LocationsMap.put("koc university", new Location( locationName: "Koç University",  latitude: 41.2026952f,  longitude: 29.0714802f));
    LocationsMap.put("sariyer", new Location( locationName: "Sariyer",  latitude: 41.1713436f,  longitude: 28.695352f));
    LocationsMap.put("hacisoman metro", new Location( locationName: "Hacisoman Metro",  latitude: 41.1395095f,  longitude: 29.0203817f));
    LocationsMap.put("istinyepark", new Location( locationName: "IstinyePark",  latitude: 41.1103935f,  longitude: 29.0229535f));
    LocationsMap.put("besiktas", new Location( locationName: "Besiktas",  latitude: 41.0716882f,  longitude: 28.9417285f));
    LocationsMap.put("levent", new Location( locationName: "Levent",  latitude: 41.0824497f,  longitude: 28.9954444f));
    LocationsMap.put("bebek", new Location( locationName: "Bebek",  latitude: 41.0776892f,  longitude: 29.0337483f));
    LocationsMap.put("sultanahmet", new Location( locationName: "Sultanahmet",  latitude: 41.0078663f,  longitude: 28.9367177f));
    LocationsMap.put("caddebostan", new Location( locationName: "Caddebostan",  latitude: 41.0078663f,  longitude: 28.9367177f));
    LocationsMap.put("bagdat street", new Location( locationName: "Bağdat Street",  latitude: 40.9668047f,  longitude: 29.0217849f));
    LocationsMap.put("ortakoy", new Location( locationName: "Ortakoy",  latitude: 40.9589491f,  longitude: 29.0732007f));
}
```

OCM Client Overview

The client provides an interactive command-line interface, allowing users to select a specific action and enter the required inputs seamlessly.

OCMClient

This is the core class of the client side. It reads the user input, and builds requests based on the input and the parameters using OCMProtocolHandler. It sends the formatted commands to the server. It then receives and prints server responses. The timeout is set to one minute, after which the connection is closed in the case of a delayed response from the server.

OCMProtocolHandler

It constructs the protocol-compliant messages. The following is an example of creating a protocol message for the STATIONS method.

```
public String createStationRequest(String type, String... args) {  3 usages
    return switch (type) {
        case "coord" -> "%s STATIONS -lat %s -long %s".formatted(protocolName, args[0], args[1]);
        case "country" -> "%s STATIONS -country %s".formatted(protocolName, args[0]);
        case "location" -> "%s STATIONS -location %s".formatted(protocolName, args[0]);
        default -> "";
    };
}
```

OCMClientUI

It contains the menu messages for introduction and station submenu.

Execution of the Program

Server is started and two clients are connected to the server simultaneously.

```
OCM Server started on port: 8000
A connection was established with a client on the address of /127.0.0.1:55115
A connection was established with a client on the address of /127.0.0.1:55117
```

The welcome message and the menu is printed for the user to choose an action.

```
Welcome to the OCM416 Server!
---------------------------
Available Services:
1. Stations - Find the nearest charging stations
2. Locations - View available locations
3. Countries - View supported countries
4. Go back
5. Exit
>
```

Invalid input displays an error message.

```
>6
Invalid entry!
```

Actions and Error Handling

1. View Available Locations

The client sends a OCM416 LOCS request to the server, and retrieves the pre-defined locations.

```
Available Services:
1. Stations - Find the nearest charging stations
2. Locations - View available locations
3. Countries - View supported countries
4. Go back
5. Exit
>2
    Bağdat Street                   Bebek                       Besiktas                    Caddebostan
    Hacisoman Metro                 IstinyePark                 Koç University             Levent
    Ortakoy                         Sariyer                     Sultanahmet
```

2. View Supported Countries

The client sends a OCM416 CTS request to the server, the server fetches the country data from the API, and sends it to the client.

```
>3
    Afghanistan                     Aland Islands               Albania                     Algeria
    American Samoa                  Andorra                     Angola                      Anguilla
    Antarctica                      Antigua And Barbuda         Argentina                   Armenia
    Aruba                           Australia                   Austria                     Azerbaijan
    Bahamas                         Bahrain                     Bangladesh                  Barbados
    Belarus                         Belgium                     Belize                      Benin
    Bermuda                         Bhutan                      Bolivia, Plurinational State Of  Bonaire, Saint Eustatius And Saba
    Bosnia And Herzegovina          Botswana                    Bouvet Island               Brazil
    British Indian Ocean Territory  Brunei Darussalam           Bulgaria                    Burkina Faso
    Burundi                         Cambodia                    Cameroon                    Canada
    Cape Verde                      Cayman Islands              Central African Republic    Chad
    Chile                           China                       Christmas Island            Cocos (Keeling) Islands
    Colombia                        Comoros                     Congo                       Congo, Democratic Republic Of The
```

3. Find the Nearest Charging Stations

a. Find the charging stations based on latitude and longitude:

```
> 1
Select the method
a. By coordinates - Enter latitude and longitude
b. By country - Enter country name
c. By location - Enter location name
d. Go back
>a
Enter latitude:
> 40
Enter longitude:
> 28

----- Charging Stations -----

ZES - 0178 İzmir Yönü DC,05 Otoyolu 178. Km Yasa Dinlenme Tesisleri İzmir Yönü,https://www.google.com/maps?q=39.9247,28.1337,4
------------------------------------------------------------------------
ZES - 0178 İstanbul Yönü DC,05 Otoyolu 178. Km Yasa Dinlenme Tesisleri İstanbul Yönü,https://www.google.com/maps?q=39.9246,28.1340,4
------------------------------------------------------------------------
```

An error message is displayed if the user gives an invalid input.

```
>a
Enter latitude:
> 1234
Enter longitude:
> 5678
Invalid input!
```

b. Find the charging stations based on country:

```
>b
Enter country name:
> germany

----- Charging Stations -----

Eifel-Camp (PowerGo),Am Freilinger See 1,https://www.google.com/maps?q=50.4161,6.7180,0
------------------------------------------------------------------------
Gutshof Rethmar,Gutsstraße 15,https://www.google.com/maps?q=52.3123,10.0020,4
```

An error message is displayed if the country is invalid.

```
>b
Enter country name:
> Hogwarts
Country not found!
```

c. Find the charging stations based on location:

```
>c
Enter location name:
> Bebek

----- Charging Stations -----

POWERŞARJ - Barbaros Hayrettin Paşa Camii,Levent, Levent Cd. No:5 D:3,https://www.google.com/maps?q=41.0788,29.0144,1
------------------------------------------------------------------------
ZES - Zorlu Center,Esentepe Mh.,https://www.google.com/maps?q=41.0677,29.0181,4
------------------------------------------------------------------------
Uğur Mumcu Caddesi,Uğur Mumcu Caddesi,https://www.google.com/maps?q=41.0879,29.0180,1
```

An error message is displayed if the location is invalid.

```
>c
Enter location name:
> Nope
Location not found or no stations available for that location!
```

4. **Exit the System**
    a. **Client Side**

```
>5
Connection Closed


Process finished with exit code 0
```

    b. **Server Side**

```
OCM Server started on port: 8000
A connection was established with a client on the address of /127.0.0.1:55839
Client disconnected: /127.0.0.1:55839
```

5. **Client inputs an empty request**

```
Available Services:
1. Stations - Find the nearest charging stations
2. Locations - View available locations
3. Countries - View supported countries
4. Go back / Display Menu
5. Exit
>
Invalid entry!
```

```
Select the method
a. By coordinates - Enter latitude and longitude
b. By country - Enter country name
c. By location - Enter location name
d. Go back
>
Invalid selection! Please enter a, b, c, or d.
```

6. **The given country does not have charging stations**

```
Select the method
a. By coordinates - Enter latitude and longitude
b. By country - Enter country name
c. By location - Enter location name
d. Go back
>b
Enter country name:
> Antarctica
Stations not found nearby, try another location!
```